
Supplementary Information: Detecting and Correcting Reasoning Failures: A Controlled Negative Result for a Residual-Stream Monitor

Arnav Kakani

Emerald High School | Algoverse AI Research
arnavkakani@gmail.com

Mrinal Agarwal

Emerald High School | Algoverse AI Research
smmrinal2009@gmail.com

Abstract

This document accompanies the main paper. It gives everything needed to rerun or extend the experiments: where the code and run artifacts are and what each file contains, the exact command and constants that produced the reported runs, the drift-score definitions as implemented, the source of the two modules that define them, the author contributions, and the compute statement. Every number in the main paper regenerates from the released run artifacts without loading a model.

1 Code, data, and artifacts

The evaluation pipeline, every run artifact, and the figure scripts are released at <https://github.com/ArnavKakani/DCRF>. The repository has four parts.

`auto_pipeline/` The pipeline. `run_all.py` runs the full chain for one model: layer selection (`stage0_layer_select.py`), threshold calibration (`stage1_threshold_sweep.py`), the four-path recovery ablation (`stage2_ablation.py`), correctness and typology labeling (`stage_labeling.py`, `failure_typology.py`, `retype_failures.py`), the threshold-versus-accuracy sweep (`stage_threshold_downstream.py`), seed robustness (`stage3_robustness.py`), the coherence judge (`stage4_judge.py`), and aggregation (`stage5_aggregate.py`, `stage_results.py`). The drift scores are in `drift_metrics.py`; model loading, prompting, sampling, rollback, the answer extractor, and the exact-match rule are in `runtime.py`; every constant is in `config.py` (Section 4).

`runs/` The eleven run directories behind every table and figure, named `<timestamp>-<model>-<dataset>-<metric>`. Each contains `manifest.json` (model, dataset, full configuration snapshot, per-stage status and timing); `trajectories.jsonl` (one line per problem: uid, per-problem maximum z -score `max_z`, trigger index `spike_idx`, sampled-baseline answer and correctness); `ablation.jsonl` (one line per triggered problem: gold answer, the four recovery continuations `path_a_baseline`, `path_b_full_cure`, `path_c_prompt_only`, `path_d_rewind_only`, trigger token and z); `labeling.csv` and `failure_types.csv` (recovery flips and the typology); `judge_results.csv` (coherence-judge rates per path); `layer_select.csv`, `layer_traces.csv`, `threshold_sweep.csv` (calibration); `threshold_downstream/` and `robustness/` (sweeps); and `master_results.csv`. The aggregated `runs/master_results.csv` and `runs/results_table.csv` are the direct sources of Table 1.

auto_pipeline/figures/ make_paper_figures.py reads only the CSV and JSONL files under runs/ and regenerates every figure and the two generated tables; it needs no GPU and no model.

pipelines/ The two standalone ablation scripts (pointwise L2 and set-to-set Sinkhorn) from which the automated stages were built.

Data. GSM8K and SVAMP are loaded from their public releases through the dataset loader in runtime.py; the first 100 test problems of each are used for the main ablation, the first 20 GSM8K problems and 20 problems each of CNN/DailyMail and MBPP for layer selection. No data is modified or re-packaged.

2 Reproducing the experiments

Install the requirements (pip install -r requirements.txt; transformer_lens, geomloss, torch), then run

```
python auto_pipeline/run_all.py --model <hf-model-id> --dataset gsm8k --metric l2,sinkhorn,random
```

where random is the random-position control. Each run writes a new directory under runs/. The GPU must hold a 7B model in bfloat16 (the weights alone are about 14 GB); by the timestamps in each run’s manifest.json, the eleven runs took between 0.5 and 2.7 wall-clock hours each on single NVIDIA H100 and A100 instances rented from Lambda. Seed 42 reproduces the sampled trajectories exactly, so the detector and random-control runs share their wrong-answer sets. To regenerate the paper’s figures and tables from the released artifacts:

```
python auto_pipeline/figures/make_paper_figures.py --out figs
```

3 Drift scores as implemented

Let $h_t^{(\ell)} \in \mathbb{R}^d$ be the residual-stream state at layer ℓ for generated token t (read from blocks.{\ell}.hook_resid_post), and $\hat{h}_t = h_t^{(\ell)} / (\|h_t^{(\ell)}\|_2 + \epsilon)$ with $\epsilon = 10^{-8}$. The first $w_H = 10$ generated states are the anchor.

$$\begin{aligned} \text{L2: } D_t &= \|\hat{h}_t - \bar{h}\|_2^2, & \bar{h} &= \frac{1}{w_H} \sum_{i=1}^{w_H} \hat{h}_i; \\ \text{Sinkhorn: } D_t &= S_\lambda \left(\frac{1}{k} \sum_{j=0}^{k-1} \delta_{\hat{h}_{t-j}}, \frac{1}{w_H} \sum_{i=1}^{w_H} \delta_{\hat{h}_i} \right), & k &= 5, \end{aligned}$$

where S_λ is the entropy-regularized 2-Wasserstein distance computed by geomloss.SamplesLoss("sinkhorn", p=2, blur=0.1). Scores are standardized within each problem against the $w_T = 5$ states after the anchor, $z_t = (D_t - \mu_{\text{tare}}) / \max(\sigma_{\text{tare}}, 10^{-5})$, and the monitor fires at the first t with $z_t > \tau$. A Sinkhorn distance between two single vectors reduces to $\|\hat{h}_t - \bar{h}\|_2^2$; only the set-to-set form above performs any transport.

4 Constants used for the reported runs

Excerpt of auto_pipeline/config.py as recorded in every manifest.json.

```
USE_CHAT_TEMPLATE = True
METRICS = ["l2", "sinkhorn", "random"]
LAYER_BAND = [18, 20, 22, 24, 26]
LAYER_SELECT_SAMPLES = 20
LAYER_SELECT_CONTROL_SAMPLES = 20
LAYER_SELECT_ROLLOUTS = 1
Z_GRID = [2.0, 4.0, 6.0, 8.0, 10.0, 12.5, 15.0]
TARGET_TPR = 0.60
```

```

MAX_FPR = 0.20
THRESHOLD_DOWNSTREAM_SAMPLES = 20
ABLATION_SAMPLES = 100
ROBUSTNESS_SEEDS = [42, 43, 44, 45, 46]
ROBUSTNESS_SAMPLES = 30
MAX_NEW_TOKENS_DETECTION = 400
MAX_NEW_TOKENS_RECOVERY = 256
REWIND_TOKENS = 15
COGNITIVE_RESET_TEXT = "\nWait, let me recalculate this step to be sure:\n"
HEALTHY_WINDOW = 10
RECENT_WINDOW = 5
TARE_WINDOW_SIZE = 5
TEMPERATURE = 0.85
TOP_K = 50
BASE_SEED = 42
JUDGE_MAX_NEW_TOKENS = 8
TARGET_DATASETS = ["gsm8k", "svamp"]
CNN_PROMPT = "Summarize: {q}\nSummary:"
MBPP_PROMPT = "System: You are an expert Python programmer.\nUser: {q}\nCode:"

```

5 Source: drift_metrics.py

The module that defines the scores above, verbatim.

```

"""Drift metrics for the automated pipeline one registry, five geometries.

Every metric exposes the same small interface so the detection engine in
runtime.py can drive any of them without special-casing:

    build_reference(healthy_norm, healthy_raw) -> ref
        Called once, after the HEALTHY_WINDOW tokens, to summarise the healthy
        regime. 'healthy_norm' / 'healthy_raw' are lists of (1, D) tensors
        (L2-normalised and raw residual states respectively).

    score(h_norm, h_raw, recent_norm, ref) -> float
        Called every post-healthy token. 'recent_norm' is the list of the most
        recent RECENT_WINDOW normalised states (only meaningful when uses_set).

Flags:
    uses_set metric needs the sliding window as a *cloud* (only scores once the
        window is full) true for Sinkhorn, false for the point metrics.
    is_control metric has no detection geometry; it is the random-position
        baseline and is handled directly by Stage 2 (skips layer/threshold
        selection). Kept in the registry so the chain stays uniform.

The four real metrics differ only in geometry; the random control differs in
*mechanism*. Together they answer "is the latent signal real, and does this
particular geometry matter?" (TODO #7 random baseline, #8 metric ablation).
"""

import torch
from geomloss import SamplesLoss

# Real entropic-OT solver; only the set-vs-set metric actually transports mass.
_sinkhorn_solver = SamplesLoss(loss="sinkhorn", p=2, blur=0.1, backend="tensorized")

class Metric:
    name = "base"
    uses_set = False
    is_control = False

    def build_reference(self, healthy_norm, healthy_raw):
        raise NotImplementedError

    def score(self, h_norm, h_raw, recent_norm, ref):
        raise NotImplementedError

class L2Metric(Metric):
    """Point-to-point squared L2 between the current state and the healthy mean.

    This is exactly what the original geomloss(h_t, v_anchor) call degenerated to
    (one point per side -> nothing to transport -> ground cost ||h - v||^2).
    """
    name = "l2"

```

```

def build_reference(self, healthy_norm, healthy_raw):
    v = torch.stack(healthy_norm).mean(dim=0)
    return v / (v.norm(p=2, dim=-1, keepdim=True) + 1e-8)

def score(self, h_norm, h_raw, recent_norm, ref):
    return torch.sum((h_norm - ref) ** 2).item()

class CosineMetric(Metric):
    """Cosine distance (1 - cos) between the current state and the healthy mean.

    Same anchor as L2 but scale-invariant: responds only to *direction* change.
    """
    name = "cosine"

    def build_reference(self, healthy_norm, healthy_raw):
        v = torch.stack(healthy_norm).mean(dim=0)
        return v / (v.norm(p=2, dim=-1, keepdim=True) + 1e-8)

    def score(self, h_norm, h_raw, recent_norm, ref):
        # both operands are unit vectors, so the dot product is the cosine
        return (1.0 - torch.sum(h_norm * ref)).item()

class ResidualNormMetric(Metric):
    """Deviation of the raw residual-stream magnitude from its healthy mean.

    A direction-free control: ignores *where* the state points and looks only at
    how large the residual got. Uses the UN-normalised states (the norm is the
    signal) the only metric that consumes h_raw.
    """
    name = "residual_norm"

    def build_reference(self, healthy_norm, healthy_raw):
        norms = torch.stack([h.norm(p=2) for h in healthy_raw])
        return norms.mean().item()

    def score(self, h_norm, h_raw, recent_norm, ref):
        return abs(h_raw.norm(p=2).item() - ref)

class SinkhornMetric(Metric):
    """Real set-vs-set entropic optimal transport.

    mu = cloud of healthy-window normalised states (kept, not averaged)
    nu_t = sliding window of the RECENT_WINDOW most recent normalised states
    D_t = Sinkhorn(nu_t, mu)

    Responds to how the *distribution* of recent states has moved, not to a single
    outlier token the only setting where OT is a meaningful choice over L2.
    Lives on a different numeric scale than the point metrics, which is fine
    because triggering is on the standardised Z-score, calibrated in Stage 1.
    """
    name = "sinkhorn"
    uses_set = True

    def build_reference(self, healthy_norm, healthy_raw):
        return torch.cat(healthy_norm, dim=0) # (w, D) cloud

    def score(self, h_norm, h_raw, recent_norm, ref):
        recent_set = torch.cat(recent_norm, dim=0) # (k, D) cloud
        return _sinkhorn_solver(recent_set, ref).item()

class RandomControlMetric(Metric):
    """Matched-position control: no geometry, trigger at a random token.

    Detection is bypassed (Stage 2 draws a uniform trigger index per problem and
    runs the same rollback+reprompt). Establishes that the latent signal beats
    intervening at a random position the highest-leverage baseline (TODO #7).
    """
    name = "random"
    is_control = True

METRICS = {
    m.name: m
    for m in (
        L2Metric(),
        CosineMetric(),
        ResidualNormMetric(),

```

```
        SinkhornMetric(),  
        RandomControlMetric(),  
    )  
}
```

6 Author contributions

Arnav Kakani designed the study, built the evaluation pipeline and the controls, ran all experiments, and performed the analysis. Mrinal Agarwal contributed to the writing and editing of the manuscript and served as the reciprocal reviewer. Both authors wrote and revised the paper.

7 Compute and acknowledgments

The GPU compute for all experiments was provided through the Algoverse AI Research program, which both authors took part in during this work; we thank the program for it. All experiments ran on single NVIDIA H100 and A100 instances rented from Lambda, one 7B model in `bfloat16` at a time, for roughly 19 wall-clock hours across the eleven runs (June 24–25, 2026). The research questions, pipeline, experiments, analysis, and writing are the authors’ own.